

Sprawozdanie Projekt 1

Obliczenia Ewolucyjne

Szymon Poznański

Hubert Chruścicki

Przemysław Wojnar

Technologie użyte w projekcie

Aplikacja została zaprojektowana w architekturze klient-serwer jako nowoczesna aplikacja webowa Full-Stack. Podział responsywności i logiki biznesowej zrealizowano przy pomocy następujących technologii:

Część serwerowa - Backend:

- **Python** – Główny język programowania wykorzystany na serwerze. Idealnie nadaje się do implementacji złożonych obliczeń matematycznych, takich jak logika algorytmów ewolucyjnych.
- **Flask** – Lekki i elastyczny framework webowy języka Python. Posłużył on do stworzenia serwera oraz zbudowania interfejsu REST API, który obsługuje żądania napływające z aplikacji klienckiej i zwraca wyniki optymalizacji.
- **Swagger UI / OpenAPI** – Narzędzie zintegrowane w ścieżce /api/docs, służące do interaktywnego podglądu, testowania i dokumentacji wystawionych punktów końcowych aplikacji backendowej.
- **Jupyter Notebook** – Wykorzystany dodatkowo do szybkiego prototypowania, weryfikacji i testowania działania samego algorytmu analitycznego w izolacji od serwera.

Część kliencka - Frontend:

- **React** – Popularna biblioteka języka JavaScript służąca do budowy interaktywnych interfejsów użytkownika. Aplikacja frontendowa dzieli widoki na wielokrotnego użytku komponenty m.in. obsługujące formularze parametryzacji algorytmu oraz sekcje prezentacji wyników optymalizacji.
- **TypeScript** – Statycznie typowany nadzbiór języka JavaScript. Zapewnia on silne typowanie danych w strukturze projektu frontendowego, co minimalizuje ilość błędów w czasie pisania kodu i znacznie poprawia jego czytelność w szerszych aplikacjach.
- **Vite** – Nowoczesne i bardzo wydajne narzędzie budujące nowej generacji dla aplikacji webowych. Gwarantuje znacznie szybszy czas uruchamiania środowiska deweloperskiego i kompilacji w porównaniu do standardowych narzędzi takich jak Webpack.
- **HTML & CSS** – Standardowe technologie wykorzystane do definicji struktury i kaskadowego stylowania widoków na stronie.

Wymagania środowiskowe

Do poprawnego uruchomienia obu warstw aplikacji (serwerowej i klienckiej) oraz pełnego wykorzystania jej funkcjonalności, środowisko uruchomieniowe musi spełniać poniższe wymagania:

Środowisko dla części backendowej

- **Zainstalowany interpreter języka Python** - Wersja 3.8 lub nowsza.
- **Menedżer pakietów PIP** - Narzędzie do instalacji wymaganych zewnętrznych pakietów dla Pythona.
- **Wirtualne środowisko** - Zaleca się uruchomienie serwera w odizolowanym środowisku (np. moduł venv), aby uniknąć konfliktów zależności z systemem operacyjnym.
- **Biblioteki zależności:** Wszystkie pakiety wymagane do działania serwera zdefiniowane są w pliku requirements.txt.

Środowisko dla części frontendowej

- **Środowisko Node.js** - Wersja 18.x lub nowsza, wymagana do działania narzędzi budujących Vite oraz uruchomienia serwera deweloperskiego.
- **Menedżer pakietów NPM** - Wymagany do pobrania lokalnych pakietów zdefiniowanych w pliku package.json, m.in. biblioteki React, Material-UI, Recharts.

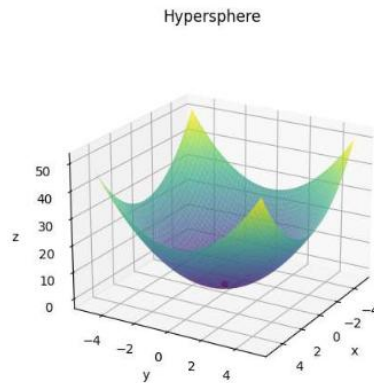
Oprogramowanie dodatkowe użytkownika

- **Przeglądarka internetowa** - Nowoczesna przeglądarka ze wsparciem dla standardu ES6 do wyświetlania interfejsu graficznego.
- **Edytor kodu** - Opcjonalnie, do modyfikacji i przeglądania kodu, posiadający zintegrowany terminal w celu łatwego zarządzania procesami.

Opis wybranych funkcji testowych i ich optima

Silnik aplikacji został zaprojektowany w sposób obiektowy, wykorzystując interfejsy i klasy abstrakcyjne, co pozwala na łatwe dodawanie nowych wariantów funkcji matematycznych. Obecnie w procesie ewaluacji algorytmu genetycznego wykorzystywana jest następująca wielowymiarowa funkcja testowa:

Funkcja Sferyczna (Hypersphere) - jest to jedna z najbardziej klasycznych, ciągłych i wypukłych funkcji testowych, powszechnie stosowana jako punkt odniesienia sprawdzający stopień zbieżności powoływanych do życia algorytmów optymalizacyjnych. Jej wykres przypomina wielowymiarową parabolę.



- **Wzór ogólny:** wartość dostosowania obliczana jest jako suma kwadratów wszystkich zmiennych decyzyjnych danego osobnika, co można opisać wzorem:

$$f(x) = \sum_{i=1}^N x_i^2$$

- **Przyjęta dziedzina poszukiwań:** każda zmienna testowa jest standardowo inicjowana w przedziale od -5.0 do 5.0.
- **Optimum - minimum globalne:** zadanie algorytmu opiera się na problemie minimalizacji. Oczekiwane, idealne i absolutne optimum globalne - najniższa możliwa wartość funkcji wynosi dokładnie 0.0 jako wartość samej funkcji (fitness), a odnajdywane jest tylko w jednym punkcie środkowym układu współrzędnych, dla wektora decyzji w którym wszystkie składowe przyjmują zerowe wartości precyzyjne: $x = [0.0, 0.0, \dots, 0.0]$

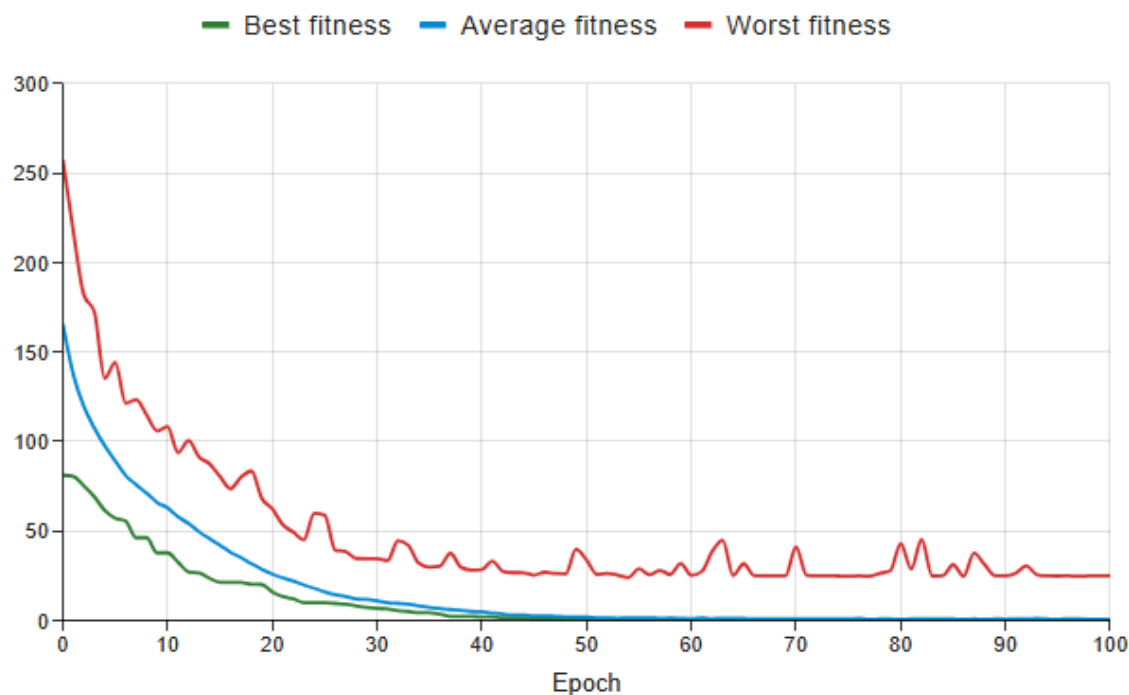
Z racji precyzji obliczeń numerycznych, dobrym i oczekiwanym rezultatem pracy algorytmu ewolucyjnego jest moment, w którym populacja we wszystkich wymiarach zbliża się możliwie najbliżej wartości idealnej wynoszącej 0.0.

Wykresy wyników

Wykres 1 (Konfiguracja Zbalansowana)

Ustawienia :

- Liczba zmiennych: 20
- Precyzja: 16 bitów
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Tournament
- Metoda krzyżowania: Two Point (Prawdopodobieństwo: 0.8)
- Metoda mutacji: One Point (Prawdopodobieństwo: 0.01)
- Prawdopodobieństwo inwersji: 0.05
- Strategia elitarna: Włączona



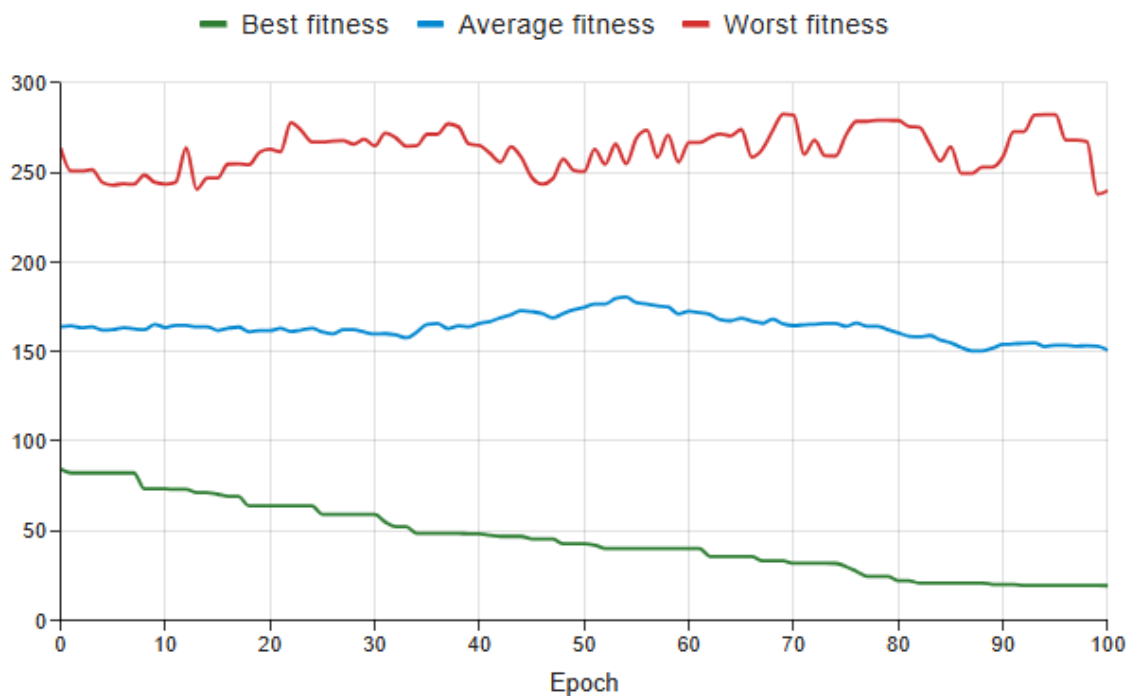
Wnioski:

- Zastosowanie selekcji turniejowej (Tournament) przy dużej populacji (500 osobników) zapewnia algorytmowi optymalny balans między eksploracją a eksploatacją.
- Na wykresie widoczna jest prawidłowa, płynna i asymptotyczna zbieżność populacji do minimum (zielona i niebieska linia opadają stabilnie).
- Niski współczynnik mutacji (0.01) wprowadza drobną, zdrową różnorodność (widoczną jako "skoki" czerwonej linii najgorszego osobnika), nie zaburzając przy tym głównego trendu optymalizacji.

Wykres 2 (Agresywna Eksploatacja)

Ustawienia :

- Liczba zmiennych: 20
- Precyzja: 16 bitów
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Best Selection
- Metoda krzyżowania: One Point (Prawdopodobieństwo: 0.85)
- Metoda mutacji: One Point (Prawdopodobieństwo: 0.01)
- Prawdopodobieństwo inwersji: 0.01
- Strategia elitarna: Włączona



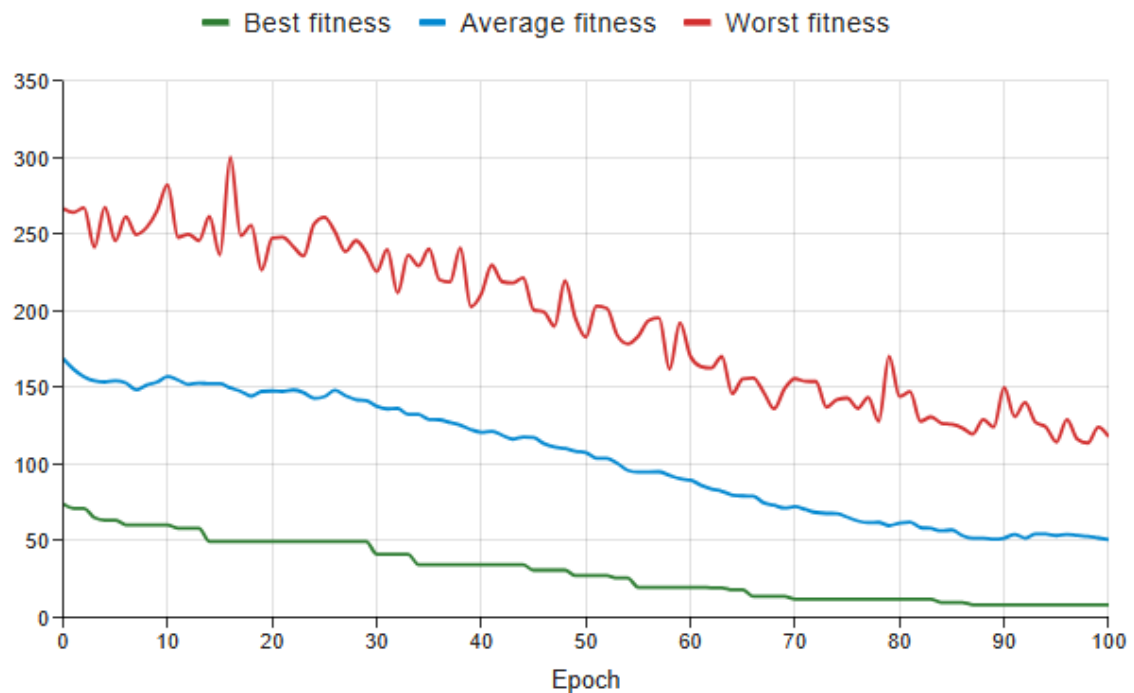
Wnioski:

- Zastosowanie metody "Best Selection" wymusza krzyżowanie wyłącznie najlepszych osobników, co prowadzi do błyskawicznej utraty różnorodności genetycznej w populacji.
- Widoczne na wykresie "schodkowanie" i wczesne wypłaszczenie linii best_fitness świadczy o zjawisku przedwczesnej stagnacji (utknięcie algorytmu w jakimś minimum lokalnym).
- Algorytm nadmiernie eksploatuje znalezione już rozwiązania, całkowicie tracąc zdolność do szerszego przeszukiwania wielowymiarowej przestrzeni.

Wykres 3 (Maksymalna Eksploracja)

Ustawienia :

- Liczba zmiennych: 20
- Precyzja: 16 bitów
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Roulette Wheel
- Metoda krzyżowania: Uniform (Prawdopodobieństwo: 0.9)
- Metoda mutacji: Two Point (Prawdopodobieństwo: 0.15)
- Prawdopodobieństwo inwersji: 0.1
- Strategia elitarna: Włączona



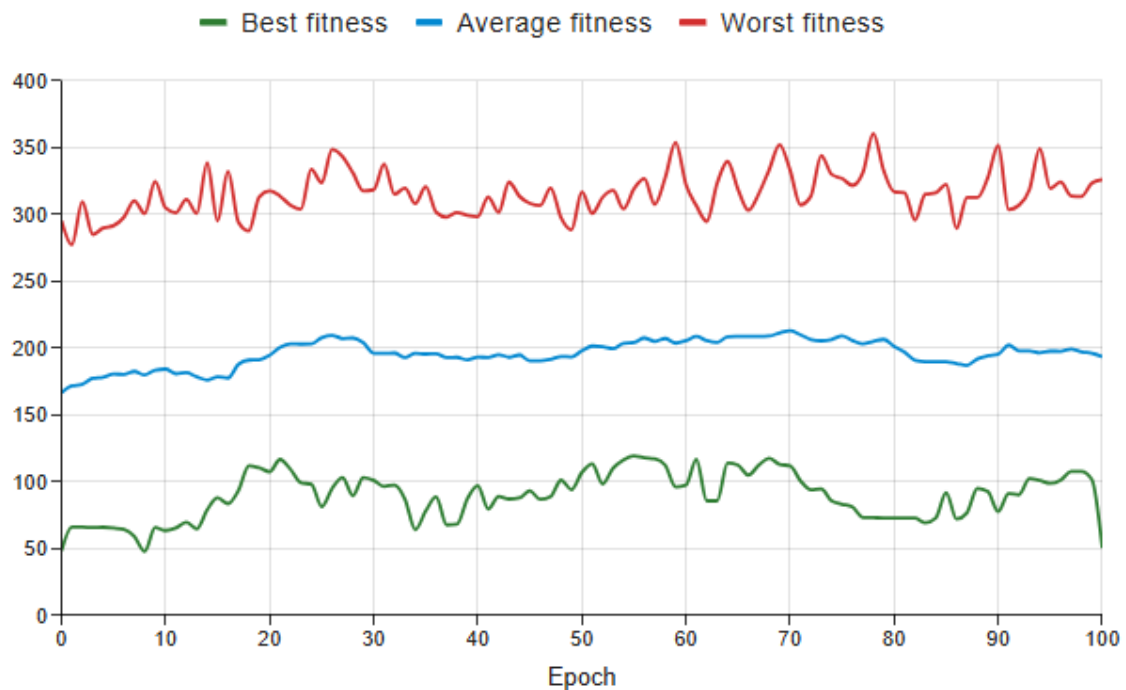
Wnioski:

- Użycie krzyżowania jednorodnego (Uniform) oraz bardzo silnej mutacji (0.15) powoduje drastyczne i ciągłe modyfikacje materiału genetycznego z epoki na epokę.
- Zdolność algorytmu do precyzyjnego dopracowywania rozwiązań (eksploatacja) jest mocno zaburzona, przez co spadek wartości best_fitness jest bardzo powolny.
- Wysoka wartość i ekstremalna niestabilność czerwonej linii (worst_fitness) dowodzą, że działanie algorytmu w dużej mierze przypomina proces błędzenia losowego, a najlepsze wyniki utrzymywane są wyłącznie dzięki strategii elitarnej.

Wykres 4 (Test bez zabezpieczeń)

Ustawienia :

- Liczba zmiennych: 20
- Precyzja: 16 bitów
- Wielkość populacji: 500
- Liczba epok: 100
- Metoda selekcji: Roulette Wheel
- Metoda krzyżowania: Two Point (Prawdopodobieństwo: 0.7)
- Metoda mutacji: Edge (Prawdopodobieństwo: 0.05)
- Prawdopodobieństwo inwersji: 0.05
- Strategia elitarna: Wyłączona



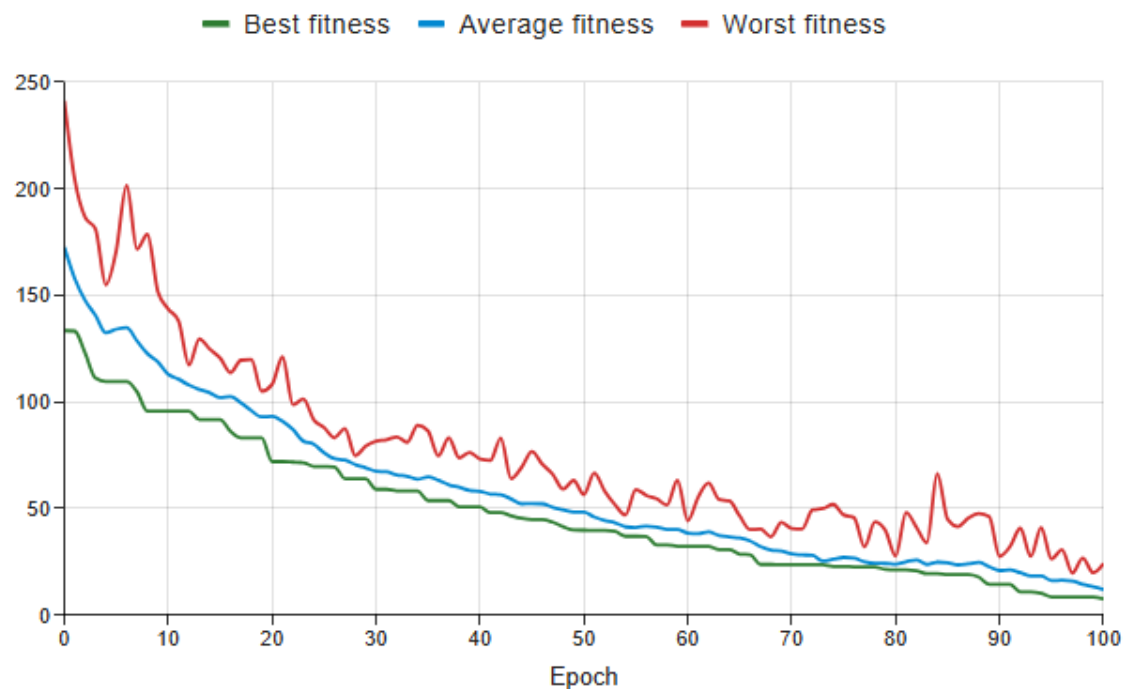
Wnioski:

- Wyłączenie strategii elitarnej w tak trudnym, 20-wymiarowym problemie okazuje się katastrofalne dla procesu optymalizacji.
- Na wykresie widać zjawisko regresji – zielona linia (best_fitness) zamiast stale maleć, wielokrotnie rośnie (pogarsza się).
- Najlepsze wypracowane dotąd osobniki nie są chronione i są bezpowrotnie niszczone przez losowe działanie krzyżowania oraz agresywną mutację brzegową (Edge Mutation).

Wykres 5 (Trudny problem, za mało osobników)

Ustawienia :

- Liczba zmiennych: 20
- Precyzja: 16 bitów
- Wielkość populacji: 40
- Liczba epok: 100
- Metoda selekcji: Roulette Wheel
- Metoda krzyżowania: Two Point (Prawdopodobieństwo: 0.8)
- Metoda mutacji: One Point (Prawdopodobieństwo: 0.02)
- Prawdopodobieństwo inwersji: 0.05
- Strategia elitarna: Włączona



Wnioski:

- Zastosowanie populacji liczącej zaledwie 40 osobników dla 20-wymiarowego problemu (każdy osobnik to ciąg aż 320 bitów) oznacza drastycznie wąską pulę genetyczną na starcie.
- Wszystkie trzy linie (best, average, worst) bardzo szybko zbliżają się do siebie, co obrazuje niemal natychmiastowe ujednolicenie się całej populacji (utrata różnorodności).
- Choć wykres opada, proces ten zachodzi wolniej i zatrzymuje się wyżej niż w konfiguracji bazowej (Wykres 1), ponieważ algorytmowi brakuje unikalnych osobników w populacji do budowy lepszych rozwiązań.

Porównanie wyników dla różnych konfiguracji algorytmu

Zestawienie wyników optymalizacji

Numer wykresu	Konfiguracja	Najlepszy wynik (Best Fitness)	Średni wynik populacji	Czas wykonania (s)
1	Konfiguracja Zbalansowana	0.0015	0.66	4.55
2	Agresywna Eksploatacja (Best Selection)	19.29	150.72	3.54
3	Maksymalna Eksploracja (Wysoka mutacja)	8.26	50.82	7.12
4	Test bez zabezpieczeń (Brak Elitaryzmu)	50.11	193.53	4.23
5	Mała populacja (40 osobników)	7.46	11.76	0.34

Analiza porównawcza i wnioski:

- Zwycięstwo zbalansowanego podejścia (Konfiguracja 1) Najlepszy wynik (0.0015) uzyskała Konfiguracja 1, co potwierdza, że w algorytmach genetycznych kluczem do sukcesu jest balans. Umiarkowana mutacja i selekcja turniejowa pozwoliły na stabilne zbliżanie populacji blisko optimum bez ryzyka utraty dobrych genów.
- Problem przedwczesnej zbieżności (Konfiguracje 2 i 5) Konfiguracje 2 (zbyt agresywna selekcja) oraz 5 (zbyt mała populacja) utknęły w minimach lokalnych z powodu szybkiej utraty różnorodności genetycznej. Algorytm przedwcześnie odrzucił słabsze, ale unikalne osobniki, odcinając sobie drogę do szerszych i skutecznych poszukiwań.
- Koszt i chaos nadmiernej eksploracji (Konfiguracja 3) Zastosowanie bardzo wysokiej mutacji (15%) i krzyżowania jednorodnego spowodowało najdłuższy czas obliczeń oraz zachowanie algorytmu przypominające losowe błędzenie. Zbyt agresywne operatory generowały ciągły szum, w którym dobre cechy były niszczone równie szybko, jak powstawały.
- Katastrofalne skutki braku elitaryzmu (Konfiguracja 4) Ta konfiguracja osiągnęła zdecydowanie najgorszy wynik (50.11), ponieważ wyłączenie strategii elitarniej pozbawiło populację ochrony wypracowanych rozwiązań. Nawet jeśli algorytm znalazł punkt bliżej zera, w kolejnej epoce był on bezpowrotnie niszczone przez losowe krzyżowanie i agresywną mutację.

Podsumowanie i analiza błędów

Uzyskane wyniki potwierdzają poprawność implementacji oraz jej zgodność z przewidywaniami teoretycznymi. Strategia elitarna stanowi warunek konieczny stabilnej zbieżności. Dobór rozmiaru populacji i presji selekcyjnej musi być proporcjonalny do wymiarowości problemu, najlepszą konfigurację charakteryzuje równowaga pomiędzy eksploracją a eksploatacją.

Analiza błędów

Błąd kwantyzacji reprezentacji binarnej

Dla przyjętej domeny $[-5, 5]$ oraz precyzji sześciu cyfr dziesiętnych pojedyncza zmienna kodowana jest na $m = 24$ bitach, co daje rozdzielczość $\Delta = (b - a)/(2^m - 1) \approx 5,96 \cdot 10^{-7}$. Wyznacza to dolną granicę osiągalnej wartości funkcji celu, której nie można przekroczyć bez zwiększenia precyzji kodowania. Jest to ograniczenie metody reprezentacji, nie implementacji.

Stochastyczny charakter procesu

Pojedyncze uruchomienie algorytmu nie stanowi miarodajnej podstawy oceny konfiguracji. Bieżąca implementacja nie udostępnia mechanizmu seedowania generatora na poziomie interfejsu, co ogranicza reprodukowalność pojedynczych wyników.

Utrata różnorodności populacji

Konfiguracje 2 i 5 ilustrują zjawisko przedwczesnej zbieżności wynikające z nieadekwatnego doboru parametrów względem wymiarowości problemu. W Konfiguracji 2 wyłączna selekcja najlepszych osobników powoduje szybką homogenizację puli genetycznej. W Konfiguracji 5 populacja 40 osobników dla ciągów 320-bitowych nie zawiera dostatecznej różnorodności początkowej.

Ograniczenia przyjętej funkcji testowej

Funkcja Hypersphere jest jednomodalna i wypukła, zatem nie weryfikuje zdolności algorytmu do przełamywania minimów lokalnych. Pełna walidacja operatorów wymagałaby zastosowania funkcji multimodalnych (Rastrigin, Ackley, Rosenbrock), dla których różnice między metodami selekcji i krzyżowania ujawniają się wyraźniej.